

Docker 2

Part 1 covered what containers are and how the pieces fit together. This part goes into the two things we work with every day: the image, and the Compose file that runs several of them.

1. What a Docker image is

Docker image is a read-only template that contains an application, its dependencies, and the instructions needed to run it.

An image is not one file. It is two things kept together:

```
Layers    → the filesystem, stacked in order
Config    → the JSON that says how to start it
```

The config is the part people forget. It carries the entrypoint, the default command, the working directory, the environment variables and the exposed ports, which is why `docker run nginx` needs no arguments.

```
docker inspect --format '{{json .Config}}' payroll:1.4
```

Output:

```
{"Env":["PATH=/usr/local/bin:/usr/bin","JAVA_HOME=/opt/java/openjdk"],
  "Cmd":null,"WorkingDir":"/app",
  "Entrypoint":["java","-jar","app.jar"],
  "ExposedPorts":{"8080/tcp":{}}}
```

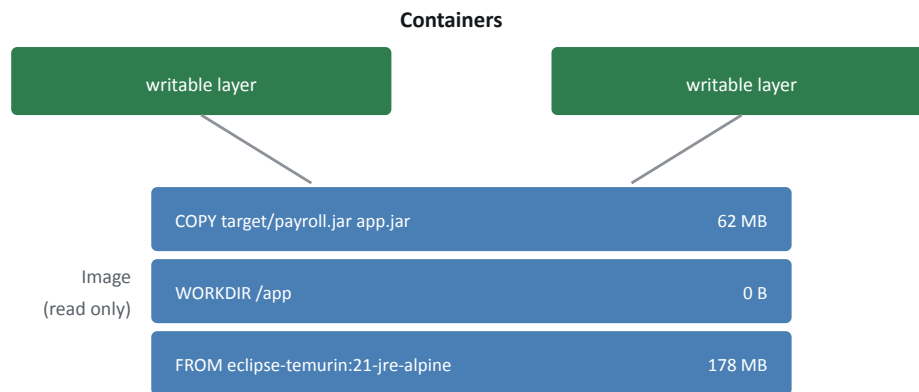
An image is identified by a digest, not by its name. The digest is a SHA-256 of the image content, so two images with the same digest are the same bytes, whatever they are called.

```
payroll:1.4 → sha256:7c9e2a1f5b3d...
```

The format is a standard, not a Docker invention. The OCI image specification is what lets Kubernetes, Podman and every registry read the same image.

2. Image layers

Each instruction in a Dockerfile that changes the filesystem adds a layer, and the layers are stacked into one view by a union filesystem, `overlay2` on Linux.



Both containers read the same 240 MB of image layers. Only the two thin writable layers belong to them.

```
docker history payroll:1.4
```

Output:

IMAGE	CREATED	CREATED BY	SIZE
7c9e2a1f5b3d	2 minutes ago	ENTRYPOINT ["java" "-jar" "app.j...]	0B
<missing>	2 minutes ago	COPY target/ payroll.jar app.jar	62MB
<missing>	2 minutes ago	WORKDIR /app	0B
<missing>	3 weeks ago	/bin/sh -c #(nop) ADD file:a3f1c8...	178MB

Only **RUN** , **COPY** and **ADD** produce a layer with size. **ENV** , **WORKDIR** , **EXPOSE** , **LABEL** , **CMD** and **ENTRYPOINT** change the config only, so they cost nothing. Merging them to "save layers" saves nothing.

Writing to a file from a lower layer copies it up first. This is copy-on-write, and it has two results worth knowing: the first write to a large file is slow, and the writable layer grows by the whole file even if one byte changed. A database writing into its writable layer is the usual way a host runs out of disk.

Layers are shared on disk. Ten images built on the same base store that base once, and ten containers from one image store the image once.

3. Image immutability

An image never changes after it is built. There is no command that edits one.

Every change produces a new image with a new digest. `docker build` on a changed Dockerfile does not update the old image; it makes another one and, if the tag is reused, moves the tag.

Four consequences follow, and all four turn up in interviews:

Rollback is repointing a tag, not rebuilding. The previous image is still in the registry, so going back means deploying the older tag. Nothing is rebuilt.

Configuration belongs at run time. A database URL baked in at build time means one image per environment. Passed in as an environment variable, one image goes everywhere.

```
docker run -e SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/payroll payroll:1.4
```

A secret added at build time is in the image forever. Deleting it in a later layer hides it from the final view and leaves it in the earlier layer, where `docker history` finds it. Use a build secret or pass it at run time.

A tag can move, a digest cannot. For anything that must be reproducible, pin the digest:

```
FROM eclipse-temurin:21-jre-alpine@sha256:4a1c8f3b9e2d7c6a5b4f3e2d1c0b9a8f
```

`docker commit` turns a running container into an image and is best avoided. The result has no Dockerfile, so nobody can say how it was made or build it again.

4. Writing a Dockerfile for Spring Boot

A working file first, then the reasoning behind each line.

```
# syntax=docker/dockerfile:1

FROM eclipse-temurin:21-jre-alpine

WORKDIR /app
COPY target/payroll.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

The first line is a parser directive. It pins the Dockerfile syntax so that newer features, such as build secrets and cache mounts, work the same way on every machine.

The build starts by sending the context to the daemon. That is the `.` at the end of `docker build -t payroll:1.4 .`, and everything in it travels before the first instruction runs.

```
target/
.git/
node_modules/
*.log
```

A `.dockerignore` with those lines is the cheapest build speed-up available, and it also stops a stray `.env` from reaching the image.

A multi-stage build is several **FROM** lines in one file. Only the last stage becomes the image, so the compiler and the source stay behind.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /src/target/ payroll.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5. Dockerfile instructions

Instruction	What it does
FROM	The base image, and the start of a stage
WORKDIR	Sets the directory for what follows. Creates it if needed
COPY	Copies files from the context into the image
ADD	Like COPY , and also unpacks archives and fetches URLs
RUN	Runs a command at build time and keeps the result
ENV	Sets an environment variable in the image and at run time
ARG	Sets a variable available at build time only
EXPOSE	Documents a port. It publishes nothing
ENTRYPOINT	The program the container runs
CMD	Default arguments, or the command when there is no entrypoint
USER	The user the following instructions and the container run as
VOLUME	Marks a path so its data is kept outside the writable layer
HEALTHCHECK	The command Docker runs to decide if the container is healthy
LABEL	Adds metadata, such as the version or the owner

Four of these cause most of the confusion.

COPY against **ADD** . **COPY** copies. **ADD** also unpacks a local tar file and downloads a URL, which is surprising behavior hiding inside an innocent line. Use **COPY** unless the archive unpacking is wanted.

ARG against **ENV**. **ARG** exists during the build and is gone afterwards. **ENV** is written into the image config and is there at run time.

```
ARG JAR_FILE=target/payroll.jar      # build only
ENV TZ=UTC                          # visible to the running application
COPY ${JAR_FILE} app.jar
```

An **ARG** is not a secret. Its value is recorded in the image history.

RUN belongs in one layer, with its cleanup. Each **RUN** is a layer, so a cleanup in a later one removes nothing.

```
# wrong: three layers, and the cache is still in layer two
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# right: one layer, nothing left behind
RUN apt-get update \
  && apt-get install -y --no-install-recommends curl \
  && rm -rf /var/lib/apt/lists/*
```

USER should not stay root. A container runs as root by default, and root inside the container is root on a mounted host directory.

```
RUN addgroup -S app && adduser -S app -G app
USER app
```

6. Image tagging

A tag is a readable label that points at an image digest. It is a pointer, not a copy.

```
registry.company.com/team/payroll:1.4.2
└── registry ───┴── repo ┴── tag ┴──
```

Part	Missing means
Registry	<code>docker.io</code>
Namespace	<code>library</code> , which is where the official images live
Tag	<code>latest</code>

So `postgres` and `docker.io/library/postgres:latest` are the same request.

```
docker build -t payroll:1.4.2 .
docker tag payroll:1.4.2 payroll:1.4
docker tag payroll:1.4.2 registry.company.com/team/payroll:1.4.2
docker push registry.company.com/team/payroll:1.4.2
```

One image now answers to three names. `docker tag` copies nothing and takes no time, because all three point at the same digest.

`latest` is not "the newest". It is the tag used when none is given, and it only moves when somebody pushes to it. An image tagged `latest` a year ago stays `latest` forever.

7. Image versioning

Tagging is the mechanism. Versioning is the policy, and a workable one uses three kinds of tag at once:

Tag	Example	Moves?	Use for
Immutable build	<code>payroll:1.4.2</code>	Never	Deploys and rollbacks
Traceable	<code>payroll:sha-9f2c1ab</code>	Never	Finding the commit that made it
Moving	<code>payroll:1.4</code> , <code>payroll:latest</code>	Yes	Convenience for people

The rule that matters: **a deploy must name an immutable tag or a digest**. If production says `payroll:latest` , two servers started an hour apart can run different code and nothing records which.

```
docker build -t payroll:1.4.2 -t payroll:sha-9f2c1ab -t payroll:latest .
```

One build, three tags, one digest. Repeat the same care on the base image:

Base image written as	Reproducible?
<code>eclipse-temurin:latest</code>	No. Anything at all
<code>eclipse-temurin:21</code>	No. Moves with every patch release
<code>eclipse-temurin:21.0.5_11-jre-alpine</code>	Yes
<code>eclipse-temurin@sha256:4a1c8f3b...</code>	Yes, and exactly

The usual compromise is a specific tag in the Dockerfile and a digest in the lock file the build produces.

8. Image size optimization

Size costs pull time on every deploy, storage in every registry, and attack surface in every container. The levers below are in order of payoff.

Choose the smaller base. This is the whole game for a Java service.

Base image	Approximate size
<code>eclipse-temurin:21</code>	450 MB
<code>eclipse-temurin:21-jre</code>	280 MB
<code>eclipse-temurin:21-jre-alpine</code>	180 MB
<code>gcr.io/distroless/java21-debian12</code>	190 MB, with no shell

Distroless has no package manager and no shell, which is good for security and awkward the first time a container needs debugging.

Use a multi-stage build, so Maven, the source and the `~/m2` cache never reach the result. This is usually a 500 MB saving on its own.

Write a `.dockerignore`. Without one, `target/` goes into the context and often into the image.

Combine `RUN` and clean up inside the same one, as in section 5. A file deleted in a later layer still occupies space in the earlier one.

Find the fat layer rather than guessing:

```
docker history --no-trunc --format '{{.Size}}\t{{.CreatedBy}}' payroll:1.4
```

Sorting that output finds the instruction responsible in a few seconds.

Build a custom runtime with `jlink` when the last 100 MB matters. It ships only the modules the application actually uses.

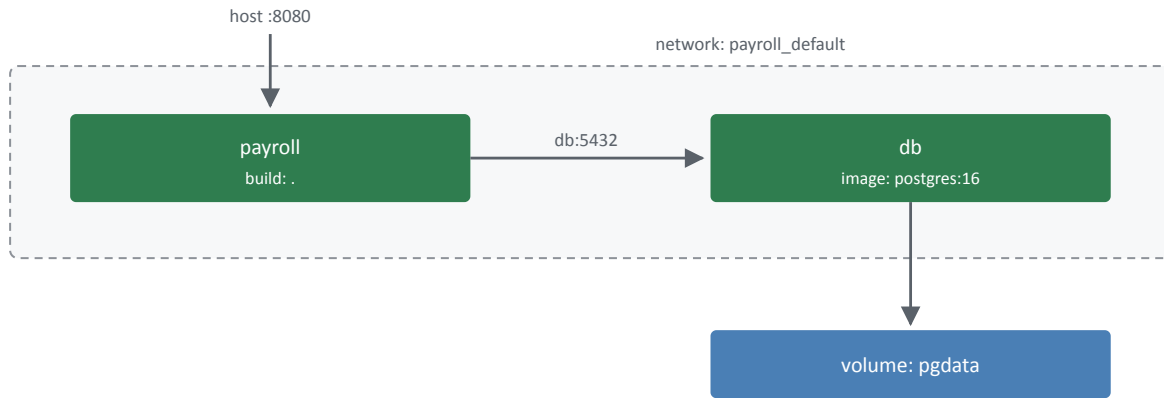
```
RUN jlink --add-modules java.base,java.logging,java.sql \
      --strip-debug --no-man-pages --compress=zip-6 \
      --output /javaruntime
```

A trimmed runtime for a typical Spring Boot service lands near 60 MB, against 180 MB for the full JRE.

9. What Docker Compose is

Docker Compose is a tool that defines and runs a multi-container application from a single YAML file.

The problem it solves is repetition. A service with a database and a cache takes three long `docker run` commands, in the right order, with the right network and the right volumes, typed correctly every time. Compose puts them in a file that lives in the repository.



One file describes all of it: the two services, the network they share, the volume that outlives them, and the one port the host can reach.

```
docker compose up -d      # build what needs building, then start everything
docker compose ps        # what is running
docker compose logs -f   # every service's output, interleaved
docker compose down      # stop and remove containers and the network
docker compose down -v   # and delete the named volumes
```

Compose is for one machine. It is the development and testing tool. Several machines is what Kubernetes is for, and saying so is usually the expected answer.

10. compose.yaml and docker-compose.yml

Both names work. The difference is history.

	Compose V1	Compose V2
Command	<code>docker-compose</code> with a hyphen	<code>docker compose</code> , a subcommand
Written in	Python, installed separately	Go, part of Docker
Preferred file	<code>docker-compose.yml</code>	<code>compose.yaml</code>
Status	Retired	Current

V2 reads either name. It looks for `compose.yaml`, then `compose.yml`, then `docker-compose.yaml`, then `docker-compose.yml`, and stops at the first one it finds.

The `version:` key is obsolete. V2 ignores it and warns about it. A file starting with `version: "3.8"` is a file written for V1, and removing that line is the whole migration for most projects.

Two more files matter:

```
compose.yaml      the base definition
compose.override.yaml applied on top automatically, for local development
.env              variable values, read automatically
```

```
docker compose -f compose.yaml -f compose.prod.yaml up -d
```

Naming the files explicitly turns the automatic override off, which is how one repository serves both local and production settings.

11. Services

A service is one container definition. Compose starts a container from it, names it after the project and the service, and can start more than one.

```
services:
  payroll:
    build:
      context: .
      dockerfile: Dockerfile
    image: payroll:1.4.2
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/payroll
    env_file:
      - .env
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:8080/actuator/health"]
      interval: 10s
      timeout: 3s
      retries: 5
      start_period: 40s
```

Key	What it does
<code>image</code>	The image to run, or the name to give the built one
<code>build</code>	Build from a Dockerfile instead of pulling
<code>ports</code>	Publish to the host, host first
<code>environment</code>	Variables for this service
<code>env_file</code>	The same, read from a file
<code>depends_on</code>	Start order, and with <code>condition</code> a readiness wait
<code>healthcheck</code>	How Compose decides the service is up

Key	What it does
<code>restart</code>	What to do when it exits
<code>command</code>	Override the image's default command

`start_period` is the one people miss. A Spring Boot service takes thirty seconds to start, so without it the healthcheck fails several times and the container is marked unhealthy before it ever had a chance.

```
docker compose up -d --scale payroll=3
```

That runs three containers from one service definition. The published port has to go first, because three containers cannot share host port 8080.

12. Networks

Compose creates one network for the project and attaches every service to it. With no configuration at all, services reach each other by service name.

```
services:
  payroll:
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/payroll
  db:
    image: postgres:16
```

`db` resolves through Docker's DNS to whatever address that container has today. No addresses are written anywhere.

The port in `ports` is for the host. The port services use is the container port. This is the most common Compose mistake:

```
ports:
  - "5433:5432"      # the host reaches the database on 5433
```

The application still connects to `db:5432`, because inside the network nothing was remapped. Only the host sees 5433.

Separate networks when parts of the stack should not reach each other:

```

services:
  web:
    networks: [frontend]
  payroll:
    networks: [frontend, backend]
  db:
    networks: [backend]

networks:
  frontend:
  backend:

```

`web` can now reach `payroll` and cannot reach `db` at all. The database is unreachable from the edge even if the web container is taken over.

A service with no `ports` is still reachable inside the network. Publishing is only about the host, so a database usually needs no `ports` entry at all.

13. Volumes

A named volume is declared at the top level and mounted into a service. It outlives the container.

```

services:
  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro

volumes:
  pgdata:

```

Two mounts, two different things:

Written as	Kind	Managed by
<code>pgdata:/var/lib/...</code>	Named volume	Docker
<code>./init.sql:/docker-...</code>	Bind mount, host path	Us

Anything starting with `.` or `/` is a bind mount. Anything else is a named volume, and it must appear under the top-level `volumes:` key or Compose refuses to start.

`:ro` mounts read-only. Configuration, certificates and seed data should all carry it.

`docker compose down` keeps named volumes. `down -v` deletes them. That one flag is the difference between restarting the stack and losing the database, so it is worth saying out loud before typing it.

```
docker compose down          # containers and network gone, data kept
docker compose down -v      # data gone as well
docker volume ls             # what still exists
```

14. How to dockerize a Spring Boot application

Start from the plain version, because it is what an interview expects first.

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/ payroll.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

It works, and it has two problems: the jar must already be built, and every code change replaces one 62 MB layer.

The multi-stage version builds inside Docker and caches dependencies.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B package -DskipTests
```

Copying `pom.xml` alone and resolving dependencies before the source arrives is the whole trick. A code change re-runs only the last line.

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY --from=build /src/target/ payroll.jar app.jar
USER app
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

A layered jar is what a production build does. Spring Boot can split its own jar into parts that change at different speeds, so a code change ships a few hundred kilobytes instead of the whole fat jar.

```
FROM eclipse-temurin:21-jre-alpine AS layers
WORKDIR /app
COPY --from=build /src/target/payroll.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract
```

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=layers /app/dependencies/ ./
COPY --from=layers /app/spring-boot-loader/ ./
COPY --from=layers /app/snapshot-dependencies/ ./
COPY --from=layers /app/application/ ./
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

Dependencies are copied first because they change least; the application classes come last because they change every commit. Spring Boot 3.3 renamed the mode to `-Djarmode=tools -jar app.jar extract --layers`, so check the version before copying this.

Pass configuration in, never build it in.

```
services:
  payroll:
    build: .
    ports: ["8080:8080"]
    environment:
      SPRING_PROFILES_ACTIVE: docker
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/payroll
      JAVA_TOOL_OPTIONS: "-XX:MaxRAMPercentage=75"
    depends_on:
      db:
        condition: service_healthy
```

Spring maps `SPRING_DATASOURCE_URL` onto `spring.datasource.url` on its own, so no code changes and no extra file are needed.

The whole stack then starts with one command, and `--build` rebuilds the image when the source has changed:

```
docker compose up --build -d
docker compose logs -f payroll
```

MaxRAMPercentage is the one JVM flag to remember. A modern JVM reads the container memory limit rather than the host's, but it defaults to a quarter of it. Setting 75 percent stops a 2 GB container from using 512 MB and then throwing `OutOfMemoryError`.

15. JAR against WAR deployment in Docker

JAR	WAR
Server is embedded inside it	Needs a server already running
<code>java -jar app.jar</code>	Dropped into the server's deploy directory
One process, container starts it	Two things to keep in step
Base image is a JRE	Base image is Tomcat or similar
Around 180 MB	Around 400 MB
The default for Spring Boot	For an existing application server

The JAR is the right answer for a container, and the reason is not size. A container is meant to run one process that starts, logs to standard output and stops on a signal. An executable jar does exactly that. A WAR adds a server with its own lifecycle, its own configuration files and its own logs inside the container.

```
# JAR: the server is in the application
FROM eclipse-temurin:21-jre-alpine
COPY target/payroll.jar /app/app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

```
# WAR: the application is in the server
FROM tomcat:10.1-jre21
COPY target/payroll.war /usr/local/tomcat/webapps/ROOT.war
```

The WAR still has its place: an organization standardized on an application server, or an application that has to sit beside others in one Tomcat. Spring Boot can produce a WAR that is also executable, which is the usual bridge while that is being unwound.

16. JDK against JRE in Docker

JDK	JRE
Compiler, tools and runtime	Runtime only
<code>javac</code> , <code>jar</code> , <code>jlink</code> , <code>jcmd</code>	None of those
Around 450 MB	Around 180 MB
The build stage	The final stage

Compile with the JDK, run with the JRE. In a multi-stage build that costs nothing, because the JDK stage is discarded.

```
FROM maven:3.9-eclipse-temurin-21 AS build      # JDK, for compiling
...
FROM eclipse-temurin:21-jre-alpine             # JRE, for running
```

Shipping the JDK adds around 270 MB and a compiler to every running container, neither of which the application uses.

Two qualifications are worth knowing:

Not every vendor still ships a JRE. Oracle stopped publishing a separate JRE after Java 11. Eclipse Temurin does publish `-jre` images, which is one reason it is the common choice for base images.

Diagnostic tools live in the JDK. `jcmd`, `jstack` and `jmap` are not in a JRE image, so a heap dump from a running container needs either a JDK image or a sidecar that has one. Teams that debug in production sometimes accept the extra size deliberately.

`jlink` settles the argument when size really matters: it uses the JDK at build time to produce a runtime smaller than any JRE, and ships only that.

The pieces together

Piece	What it answers
Layers	Why a rebuild is fast, and where the size went
Immutability	Why rollback is a tag change
Digest	How to name an exact image
<code>ARG</code> and <code>ENV</code>	Build time against run time
Tagging	What an image is called
Versioning	Which tag a deploy is allowed to name
Multi-stage	How the compiler stays out of the result
Compose file	What the whole stack looks like
Service	One container definition
Network	How services find each other
Volume	What survives <code>down</code>

For example, imagine the payroll service again. A multi-stage Dockerfile compiles with the Maven JDK image and copies the extracted Boot layers into a JRE image, so dependencies and application classes sit in separate layers and a commit ships a small one. The build tags the result `1.4.2`, `sha-9f2c1ab` and `latest` in one

command, and the deploy names `1.4.2`. Compose runs it beside PostgreSQL on the project network, where it connects to `db:5432`, and the database writes to the `pgdata` volume that `docker compose down` leaves alone.

A good way to make this stick is to take one Spring Boot application, build it three ways, and run `docker image ls` after each: the plain jar on a full JDK, the same jar on a JRE, and the layered build. The numbers make the argument better than any explanation.